File Upload Architecture

permission from the API, bytes to storage

Watch the 4-min video

In one line: don't stream uploads through your API: it issues a short-lived signed URL and the client sends bytes straight to object storage; then storage events drive validation, a malware scan in quarantine and processing, and a status field in the database tells the app when the file is ready.

1 Your API servers as byte pumps

A user uploads a 2 GB video through `POST /videos`. On a 5 Mbit/s mobile uplink that takes 2 × 8,000 ÷ 5 ≈ **53 minutes**, and for all of it an API worker holds an open connection, buffers or streams the bytes, and forwards them to storage. A thousand such uploads at once means a thousand busy workers and twice the bandwidth (in and out), and API servers now need scaling for uploads rather than requests. Managed front doors also refuse it: API Gateway caps request bodies at 10 MB and Lambda's synchronous payload at 6 MB.

2 Direct upload

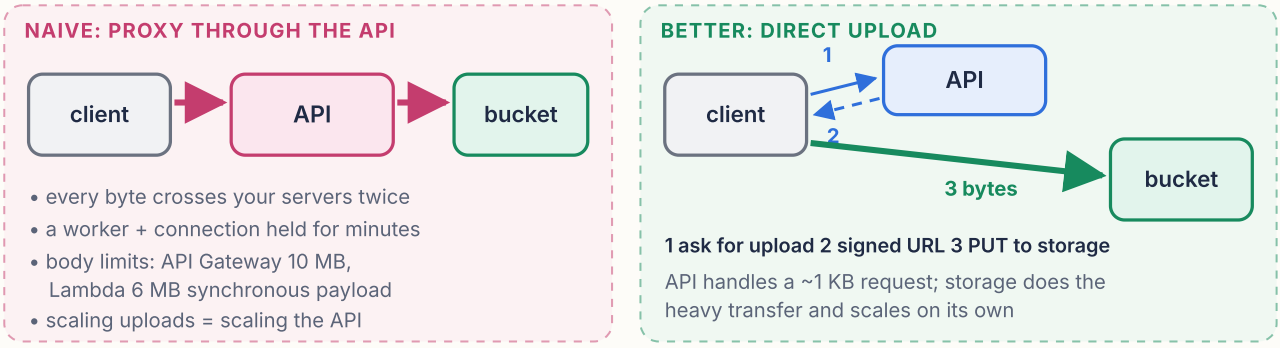


Figure 1. Proxying makes the API a byte pump. A direct upload keeps the API in the control path only.

1. **Initiate**: `POST /uploads {fileName, size, contentType}`. The API authenticates the user, checks quota, file size and allowed types, creates a `files` row with `status=pending`, and picks the object key itself (`quarantine/7f3c…`).
2. **Sign**: it returns a pre-signed URL (lesson 5) that allows exactly one PUT to that key for a few minutes. With an S3 pre-signed **POST** the policy can also enforce `content-length-range` and a `Content-Type`; Azure uses a SAS token with write-only permission.
3. **Upload**: the client sends bytes straight to storage. Files over ~100 MB use multipart or resumable uploads (lesson 6).
4. **Complete**: storage emits an `ObjectCreated` event; the client may also call `POST /uploads/{id}/complete` for a fast UI, but the server verifies with a `HEAD` rather than trusting the client.

CORS must be configured on the bucket for browser uploads (allowed origin, `PUT`, the headers you sign).

Approach	When	Watch out for
Proxy through API	tiny files (avatars < 1 MB) where you must inspect bytes inline	body limits, held workers
Pre-signed PUT	single files up to a few hundred MB	size not enforced unless signed; no resume
Pre-signed POST (policy)	browser forms needing size/type limits	S3-specific; single request
Multipart / resumable	large files, mobile networks	abandoned parts cost money (lesson 12)

3 Validate it

An upload isn't done when the bytes land. Treat every file as hostile until checked, following the OWASP File Upload Cheat Sheet:

- **Size**: enforce at signing (signed length or POST policy) and re-check the real size from the event or a `HEAD`.
- **Real type**: read the first bytes ("magic numbers"), not the extension or the client's `Content-Type`. JPEG starts `FF D8 FF`, PNG `89 50 4E 47`, PDF `%PDF-`. Allowlist types; reject everything else.
- **Deep checks**: limit image pixel counts and archive expansion (decompression bombs); re-encode images, which also strips hidden payloads and metadata.
- **Safe serving**: keep the generated key, store the user's file name only as metadata, serve downloads with `Content-Disposition: attachment` and `X-Content-Type-Options: nosniff`, ideally from a separate domain, so an uploaded HTML or SVG file can't run script on your main site.

4 Scan it in quarantine

Uploads land in a **quarantine** bucket or prefix that no user and no CDN can read. A scanner checks each object; clean files are copied (or tagged) to the clean location, infected ones are deleted or moved to an isolated bucket, and the row is marked `rejected`. Options: **ClamAV** in a container or function, **Amazon GuardDuty Malware Protection for S3**, or **Microsoft Defender for Storage** on-upload scanning; the managed ones can tag the object with the verdict. Limits to know: signature scanners miss new malware, password-protected archives can't be scanned (reject them or flag them), and very large files may exceed a scanner's size limit.

5 Driven by events

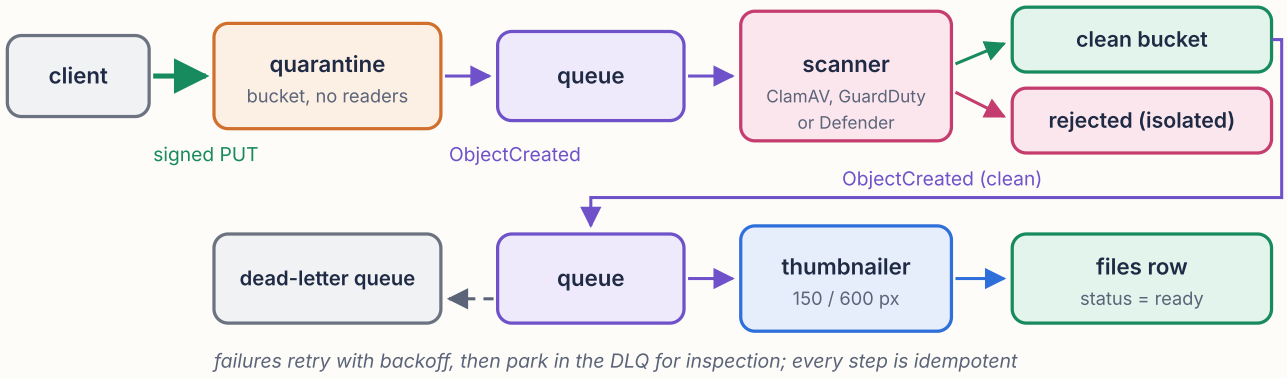


Figure 2. The worked example as a pipeline: each step is triggered by a storage event through a queue, retries on its own and scales on its own.

- **Sources**: S3 Event Notifications (to SQS, SNS, Lambda or EventBridge); Azure Event Grid `BlobCreated`; GCS Pub/Sub notifications.
- **At-least-once**: events can be duplicated and arrive out of order, so consumers must be **idempotent**: key work by object key + version (or ETag), and use conditional updates such as `UPDATE files SET status='clean' WHERE id=? AND status='scanning'`.
- **Queues between steps**: put a queue in front of each worker so a slow scanner doesn't drop events; retry with backoff and send poison messages to a dead-letter queue.
- **Reconcile**: events can be lost by misconfiguration; a periodic sweep finds rows stuck in `uploaded` or `scanning` and objects with no row, and a lifecycle rule deletes stale quarantine objects (lesson 12).

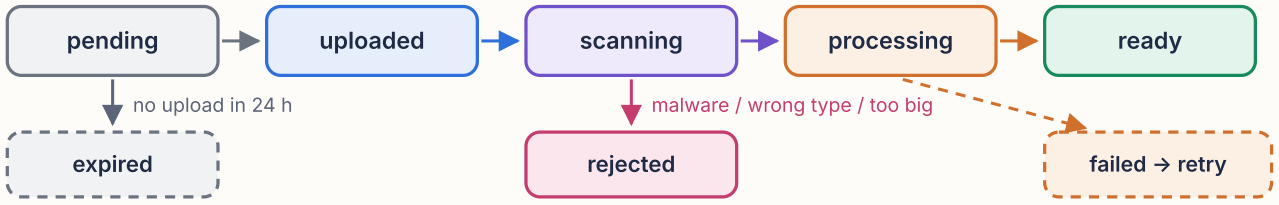


Figure 3. The upload as a state machine in the database. The app shows the file only in **ready**; every transition is a conditional update.

6 Example: upload → event → scan → thumbnail

WORKED EXAMPLE: A PHOTO UPLOAD

The app calls `POST /uploads`; the API creates row 81 (`pending`) and returns a 5-minute signed PUT URL for `quarantine/81-7f3c.jpg`, limited to 20 MB. The phone uploads the 4 MB photo directly. The bucket emits `ObjectCreated` onto a queue; the row becomes `scanning`. The scanner reads the magic bytes (JPEG), checks the pixel count, runs the malware scan, copies the object to `media/81.jpg` in the clean bucket and deletes the quarantine copy. That write emits another event; the thumbnail worker creates 150 px and 600 px versions (lesson 9) and sets the row to `ready` with the variant keys. The app, polling or notified over a WebSocket, shows the photo about 3–5 seconds after the upload finished. If the scanner had found malware, the row would read `rejected` and the user would see why.

TRADE-OFFS

- Direct uploads offload bandwidth, but you lose inline inspection; all checks become asynchronous.
- Asynchronous pipelines scale and retry well, but the file isn't usable instantly; the UI needs a "processing" state.
- Copying from quarantine to clean doubles writes briefly; tagging in place is cheaper but relies on every reader checking the tag.
- Managed scanners are simple but priced per GB scanned; self-hosted ClamAV is cheap but needs signature updates and scaling.

COMMON MISTAKES

- Trusting the extension or `Content-Type` sent by the client.
- Letting users read files before the scan finishes (e.g. serving the quarantine bucket via the CDN).
- Non-idempotent consumers: a duplicate event creates two thumbnails or charges quota twice.
- Serving user uploads from the main domain inline, turning an uploaded SVG/HTML file into stored XSS.

7 Key terms

Direct upload: client sends bytes to storage with a signed URL

Magic bytes: file signature in the first bytes; reveals the real type

Idempotent consumer: processing the same event twice has no extra effect

Quarantine: unreadable landing area until checks pass

Event notification: storage emits an event when an object is created or deleted

Dead-letter queue: where messages go after repeated failures

8 Interview questions

Q1. Why avoid streaming large uploads through your API servers? BASIC

They hold connections, memory and bandwidth for minutes, hit request-size limits, and scaling uploads means scaling the whole API; direct uploads with signed URLs send bytes straight to storage.

Q2. What happens after the upload? BASIC

Validation (size, real type), a malware scan in quarantine and processing steps such as thumbnails, triggered by storage events through queues, with a status row that flips to ready.

Q3. How do you stop a user uploading a 50 GB file through a signed URL meant for avatars? SCENARIO

Sign the size: an S3 pre-signed POST policy with content-length-range (or a signed Content-Length), check quota before signing, and re-check the real size from the event and delete oversize objects.

Q4. The thumbnail worker sometimes runs twice for the same photo. Why, and is it a problem? SCENARIO

Storage events are at-least-once, and queue redelivery adds more; make the worker idempotent (deterministic output keys, conditional status updates) and duplicates are harmless.

Q5. How does the client know when its file is ready? SCENARIO

The upload returns an ID; the client polls GET /uploads/{id} or gets a WebSocket/push notification when the pipeline sets the row to ready or rejected.

Q6. A user uploads an SVG profile picture. What can go wrong? TRICKY

SVG can contain script; served inline from your domain it's stored XSS. Rasterise or sanitise it, or serve user files as attachments from a separate domain with nosniff.

Q7. Why not trust the client's 'upload complete' call? TRICKY

It may be missing, early, faked or duplicated; verify with storage (HEAD or the ObjectCreated event) before moving the row forward.

Next: SD-O-005 · Pre-Signed URLs backendroadmap.com